Collection Theory Question

◈ Collection Interface

1. What is the Collection interface in Java?
2. How is Collection different from Collections?
3. What are the main methods of the Collection interface?
4. When should you use Collection over arrays?
5. Why is Collection interface considered the root of the framework?

◈ Iterator

6. What is an Iterator?
7. How does Iterator work internally?
8. What methods are available in Iterator?
9. When should you use Iterator instead of for-each loop?
10. Why does Iterator allow removal of elements safely?

◈ ListIterator

11. What is ListIterator?
12. How is ListIterator different from Iterator?
13. What additional methods does ListIterator provide?
14. When should you use ListIterator?
15. How can you traverse a list in reverse using ListIterator?

◈ Enumeration

16. What is Enumeration?
17. How is Enumeration different from Iterator?
18. When is Enumeration used in modern Java?
19. What are the methods of Enumeration?
20. Why is Enumeration considered legacy?

◈ List Interface

21. What is a List in Java?
22. How does List maintain insertion order?
23. When should you use List over Set?
24. Why does List allow duplicates?
25. What are common implementations of List?

◈ ArrayList

26. What is ArrayList?
27. How does ArrayList grow dynamically?
28. When should you use ArrayList?
29. Why is ArrayList faster for random access?
30. What are the disadvantages of ArrayList?

◈ LinkedList

31. What is LinkedList?
32. How is LinkedList different from ArrayList?
33. When should you prefer LinkedList?
34. Why is insertion faster in LinkedList?
35. What are the drawbacks of LinkedList?

◈ Vector

36. What is Vector?
37. How is Vector different from ArrayList?
38. When should you use Vector?
39. Why is Vector considered legacy?
40. What is synchronization in Vector?

◈ Stack

41. What is Stack in Java?
42. How does Stack follow LIFO principle?

43. What are the main methods of Stack?
44. When should you use Stack?
45. Why is Stack considered legacy?

◈ Set Interface

46. What is a Set?
47. Why does Set not allow duplicates?
48. What are different implementations of Set?
49. When should you use Set instead of List?
50. How does Set ensure uniqueness?

◈ HashSet

51. What is HashSet?
52. How does HashSet store elements internally?
53. When should you use HashSet?
54. Why does HashSet not maintain order?
55. What is the role of hashCode() and equals() in HashSet?

◈ LinkedHashSet

56. What is LinkedHashSet?
57. How is LinkedHashSet different from HashSet?
58. When should you use LinkedHashSet?
59. How does LinkedHashSet maintain insertion order?
60. Why is LinkedHashSet slower than HashSet?

◈ TreeSet

61. What is TreeSet?
62. How does TreeSet maintain sorted order?
63. When should you use TreeSet?
64. What is the difference between Comparable and Comparator in TreeSet?
65. Why does TreeSet not allow null values?

MCQ Questions

Java Collection Framework – Output MCQs (1–30)

1.

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
System.out.println(list.get(1));
```

A. 10
B. 20
C. Error
D. Null

Answer: B
Explanation: Index 1 → second element → 20

2.

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
list.add(1);
System.out.println(list.size());
```

A. 2
B. 3
C. 1
D. Error

Answer: B
Explanation: List allows duplicates

3.

```
Set<Integer> set = new HashSet<>();
set.add(1);
set.add(2);
set.add(1);
System.out.println(set.size());
```

A. 1
B. 2
C. 3
D. Error

Answer: B
Explanation: Duplicate removed

4.

```
Set<Integer> set = new LinkedHashSet<>();
set.add(3);
set.add(1);
set.add(2);
System.out.println(set);
```

A. [1,2,3]
B. [3,1,2]
C. Random
D. Error

Answer: B
Explanation: Maintains insertion order

5.

```
Set<Integer> set = new TreeSet<>();
set.add(5);
set.add(2);
set.add(8);
System.out.println(set);
```

A. [5,2,8]
B. [2,5,8]
C. Random
D. Error

Answer: B
Explanation: Sorted order

6.

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.remove("A");
System.out.println(list);
```

A. [A,B]
B. [B]
C. [A]
D. Error

Answer: B

7.

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.remove(1);
System.out.println(list);
```

A. [10]
B. [20]
C. [10,20]
D. Error

Answer: A
Explanation: Removes index 1

8.

```
Iterator<Integer> it = Arrays.asList(1,2,3).iterator();
while(it.hasNext()){
   System.out.print(it.next());
}
```

A. 123
B. 321
C. Error
D. Null

Answer: A

9.

```
List<Integer> list = Arrays.asList(1,2,3);
for(Integer i : list){
   System.out.print(i);
}
```

A. 123
B. 321
C. Error
D. Null

Answer: A

10.

```
List<Integer> list = new LinkedList<>();
list.add(1);
list.add(2);
System.out.println(list.getFirst());
```

A. 1
B. 2
C. Error
D. Null

Answer: A

11.

```
LinkedList<Integer> list = new LinkedList<>();
list.add(1);
list.add(2);
list.addFirst(0);
System.out.println(list);
```

A. [1,2]
B. [0,1,2]
C. [2,1,0]
D. Error

Answer: B

12.

```
Vector<Integer> v = new Vector<>();
v.add(1);
v.add(2);
System.out.println(v);
```

A. [1,2]
B. Error
C. Null
D. 12

Answer: A

13.

```
Stack<Integer> st = new Stack<>();
st.push(1);
st.push(2);
System.out.println(st.peek());
```

A. 1
B. 2
C. Error
D. Null

Answer: B

14.

```
Stack<Integer> st = new Stack<>();
st.push(10);
st.push(20);
st.pop();
System.out.println(st);
```

A. [10]
B. [20]
C. []
D. Error

Answer: A

15.

```
Set<String> set = new HashSet<>();
set.add("A");
set.add("B");
System.out.println(set.contains("A"));
```

A. true
B. false
C. Error
D. Null

Answer: A

16.

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
list.clear();
System.out.println(list.size());
```

A. 2
B. 1
C. 0
D. Error

Answer: C

17.

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
System.out.println(list.isEmpty());
```

A. true
B. false
C. Error
D. Null

Answer: B

18.

```
Set<Integer> set = new HashSet<>();
System.out.println(set.isEmpty());
```

A. true
B. false
C. Error
D. Null

Answer: A

19.

```
List<Integer> list = Arrays.asList(1,2,3);
list.set(1, 10);
System.out.println(list);
```

A. [1,10,3]
B. Error
C. [1,2,3]
D. Null

Answer: A

20.

```
List<Integer> list = Arrays.asList(1,2,3);
list.add(4);
```

A. Works
B. Error
C. Adds 4
D. Null

Answer: B
Explanation: Fixed-size list

21.

```
Set<Integer> set = new TreeSet<>();
set.add(10);
set.add(5);
System.out.println(set.first());
```

A. 10
B. 5
C. Error
D. Null

Answer: B

22.

```
Set<Integer> set = new TreeSet<>();
set.add(10);
set.add(5);
System.out.println(set.last());
```

A. 10
B. 5
C. Error
D. Null

Answer: A

23.

```
List<Integer> list = new LinkedList<>();
list.add(1);
list.add(2);
System.out.println(list.remove());
```

A. 1
B. 2

C. Error
D. Null

Answer: A

24.

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
System.out.println(list.indexOf(2));
```

A. 1
B. 2
C. 0
D. Error

Answer: A

25.

```
Set<Integer> set = new HashSet<>();
set.add(1);
set.add(2);
System.out.println(set.remove(1));
```

A. true
B. false
C. Error
D. Null

Answer: A

26.

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
Collections.reverse(list);
System.out.println(list);
```

A. [1,2]
B. [2,1]
C. Error
D. Null

Answer: B

27.

```
List<Integer> list = new ArrayList<>();
list.add(3);
list.add(1);
Collections.sort(list);
System.out.println(list);
```

A. [3,1]
B. [1,3]
C. Error
D. Null

Answer: B

28.

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
list.add(3);
list.remove(Integer.valueOf(2));
System.out.println(list);
```

A. [1,3]
B. [1,2]
C. [2,3]
D. Error

Answer: A

29.

```
Set<Integer> set = new HashSet<>();
set.add(1);
set.add(2);
System.out.println(set.contains(3));
```

A. true
B. false
C. Error
D. Null

Answer: B

30.

```
List<Integer> list = new ArrayList<>();
list.add(10);
```

```
list.add(20);
System.out.println(list.subList(0,1));
```

A. [10]
B. [20]
C. [10,20]
D. Error

Answer: A

Coding Questions

□ Stack-Based Questions

---

1. Valid Parentheses

Description:
You are given a string containing brackets (), {}, []. You must check whether every opening bracket has a correct closing bracket in the correct order.

Explanation:
Use stack logic:

- **Push opening brackets**
- **Pop when matching closing bracket appears**

Example:
Input → "()[]{}"
Output → true

Input → "("
Output → false

---

2. Min Stack

Description:
Design a stack that can return the minimum element at any time in constant time.

Explanation:
Track minimum values along with elements so you can always retrieve the smallest efficiently.

Example:
Operations → push(3), push(1), push(2)
Minimum → 1

---

3. Next Greater Element

Description:
For every element, find the next greater element on its right side.

Explanation:
Use stack to track elements whose next greater is not yet found.

Example:
Input → [4, 5, 2, 10]
Output → [5, 10, 10, -1]

---

4. Daily Temperatures

Description:
For each day, find how many days you must wait to get a warmer temperature.

Explanation:
Use stack to track indices of days and compare temperatures.

Example:
Input → [30, 40, 35, 50]
Output → [1, 2, 1, 0]

---

5. Reverse a Stack

Description:
Reverse all elements in a stack without using another data structure.

Explanation:
Use recursion or repeated insert-at-bottom logic.

Example:
Input → [1, 2, 3]
Output → [3, 2, 1]

---

6. Evaluate Reverse Polish Notation

Description:
Evaluate mathematical expression written in postfix form.

Explanation:

- **Push numbers**
- **Apply operation when operator appears**

Example:
Input → ["2", "1", "+", "3", "*"]
Output → 9

---

7. Largest Rectangle in Histogram

Description:
Find the largest rectangular area possible in a histogram.

Explanation:
Use stack to track heights and calculate area efficiently.

Example:
Input → [2,1,5,6,2,3]
Output → 10

□ ArrayList-Based Questions

---

8. Remove Duplicates from Sorted Array

Description:
Remove duplicate elements from a sorted list and return the new size.

Explanation:
Since the array is sorted, duplicates appear next to each other.

Example:
Input → [1,1,2,2,3]
Output → [1,2,3]

---

9. Rotate Array

Description:
Rotate array elements to the right by k steps.

Explanation:
Elements shift and wrap around.

Example:
Input → [1,2,3,4,5], k=2
Output → [4,5,1,2,3]

---

10. Find All Pairs with Given Sum

Description:
Find all pairs of elements whose sum equals a target.

Example:
Input → [1,2,3,4,5], target=5
Output → (1,4), (2,3)

---

11. Merge Two Sorted Lists (ArrayList)

Description:
Merge two sorted lists into one sorted list.

Example:
Input → [1,3,5], [2,4,6]
Output → [1,2,3,4,5,6]

---

12. Find Second Largest Element

Description:
Find the second highest number in a list.

Example:
Input → [10, 20, 30, 40]
Output → 30

---

13. Move Zeros to End

Description:
Move all zero values to the end while keeping other elements in order.

Example:
Input → [0,1,0,3,12]
Output → [1,3,12,0,0]

---

14. Find Missing Number

Description:
Find the missing number from range 0 to n.

Example:
Input → [0,1,3]
Output → 2

□ LinkedList-Based Questions

---

15. Reverse a Linked List

Description:
Reverse the direction of a singly linked list.

Explanation:
Change pointers so that next node becomes previous.

Example:
Input → 1 → 2 → 3
Output → 3 → 2 → 1

---

16. Detect Cycle in Linked List

Description:
Check if linked list has a loop.

Explanation:
Use slow and fast pointer technique.

Example:
If node points back → cycle exists
Output → true

---

17. Merge Two Sorted Linked Lists

Description:
Merge two sorted linked lists into one sorted list.

Example:
Input → 1→3→5 and 2→4→6
Output → 1→2→3→4→5→6

---

18. Find Middle of Linked List

Description:
Return middle node of linked list.

Explanation:
Use slow and fast pointers.

Example:
Input → 1→2→3→4→5
Output → 3

---

19. Remove Nth Node from End

Description:
Remove nth node from the end of list.

Example:
Input → 1→2→3→4→5, n=2
Output → 1→2→3→5

---

20. Palindrome Linked List

Description:
Check if linked list reads same forward and backward.

Example:
Input → 1→2→2→1
Output → true